

计算机集成制造系统
Computer Integrated Manufacturing Systems
ISSN 1006-5911, CN 11-5946/TP

《计算机集成制造系统》网络首发论文

题目：基于“构造-学习-强化”的超启发式算法求解三维装箱问题
作者：徐翔斌，马国飞
DOI：10.13196/j.cims.2025.0368
收稿日期：2025-10-17
网络首发日期：2026-04-07
引用格式：徐翔斌，马国飞. 基于“构造-学习-强化”的超启发式算法求解三维装箱问题[J/OL]. 计算机集成制造系统. <https://doi.org/10.13196/j.cims.2025.0368>



网络首发：在编辑部工作流程中，稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定，且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式（包括网络呈现版式）排版后的稿件，可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定；学术研究成果具有创新性、科学性和先进性，符合编辑部对刊文的录用要求，不存在学术不端行为及其他侵权行为；稿件内容应基本符合国家有关书刊编辑、出版的技术标准，正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性，录用定稿一经发布，不得修改论文题目、作者、机构名称和学术内容，只可基于编辑规范进行少量文字的修改。

出版确认：纸质期刊编辑部通过与《中国学术期刊（光盘版）》电子杂志社有限公司签约，在《中国学术期刊（网络版）》出版传播平台上创办与纸质期刊内容一致的网络版，以单篇或整期出版形式，在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊（网络版）》是国家新闻出版广电总局批准的网络连续型出版物（ISSN 2096-4188，CN 11-6037/Z），所以签约期刊的网络版上网络首发论文视为正式出版。

DOI:10.13196/j.cims.2025.0368

基于“构造-学习-强化”的超启发式算法求解三维装箱问题

徐翔斌⁺, 马国飞

(华东交通大学 综合立体交通信息感知与融合江西省重点实验室, 江西 南昌 330013)

摘要: 三维装箱问题是典型的 NP 难问题, 传统启发式方法依赖经验规则与大量参数调优, 泛化与稳定性不足。针对这一难题, 提出一种基于“构造-学习-强化”的混合超启发式算法, 对三维装箱问题的装箱顺序与放置策略进行联合优化。首先设计了“两层三段式”的高层控制策略: 用贪婪随机自适应搜索算法生成受限候选表, 以 Q-learning 在受限候选表上进行 ϵ -贪婪自适应选择, 并通过路径重连算法在算子序列空间上进行搜索; 其次设计了九种扰动算子以增强搜索的多样性, 最后采用最深左下角放置策略与三空间分割策略来完成最终的货物装载。算法在 Martello 随机算例上取得接近于最优解的结果, 平均差距仅为 0.97%, 在 LN 算例上求解结果均优于对比算法, 并在两组企业实例中分别实现 99.45% 的和 92.68% 的装载率。结果表明, 与传统启发式方法相比, 该算法能有效提高三维装箱问题的求解质量和稳定性, 并且具备良好的可迁移性, 可应用于求解其他类似的组合优化问题。

关键词: 三维装箱; 超启发式算法; 贪婪随机自适应搜索算法; Q 学习; 路径重连算法

中图分类号: TP18 **文献标识号:** A

Construction-learning-intensification hyper-heuristic for 3D bin packing

XU Xiangbin⁺, MA Guofei

(Jiangxi Provincial Key Laboratory of Comprehensive Stereoscopic Traffic Information

Perception and Fusion, East China Jiaotong University, Nanchang 330013, China)

Abstract: The three-dimensional bin packing problem (3D-BPP) is an NP-hard combinatorial optimization problem. This paper proposes a hybrid hyper-heuristic under a construction-learning-intensification framework to jointly optimize the packing sequence and placement strategy for 3D-BPP. At the upper level, a two-level, three-stage control strategy is designed: a Greedy Randomized Adaptive Search Procedure (GRASP) generates a restricted candidate list, Q-learning performs ϵ -greedy adaptive operator selection on

收稿日期: 2025-10-17; 修订日期: 2026-03-04。Received 17 Oct. 2025; accepted 04 Mar. 2026.

基金项目: 国家自然科学基金资助项目(72261011)。**Foundation item:** Project supported by the National Natural Science Foundation, China (No. 72261011).

this list, and path relinking (PR) searches in the operator-sequence space. At the lower level, nine perturbation operators are constructed to enhance search diversity, and a deepest-bottom-left placement strategy together with a three-space splitting strategy is used to obtain the final loading plan. The algorithm is evaluated on the random instances of Martello and on the Loh & Nee (LN) instances, and is compared with several existing algorithms. On the Martello instances, the average gap from the best-known solutions is 0.97%. On the LN instances, all solutions are better than those of the compared algorithms, and two industrial cases reach loading rates of 99.45% and 92.68%, respectively. The results show that, compared with traditional heuristic methods, solution quality and stability for three-dimensional bin packing are improved, and good transferability to other related combinatorial optimization problems is demonstrated.

Keywords: three-dimensional bin packing; hyper-heuristic; greedy randomized adaptive search procedure; Q-learning; path-relinking

1 引言

装箱问题是典型 NP 难问题^[1], 一直受到学术界和工业界的广泛关注。三维装箱问题指的是将一组具有三维形状的货物完全装载到给定的箱子中, 要求货物完全位于箱子内部, 并且货物之间没有重叠^[2], 目标是尽可能地减少箱子的空间浪费。三维装箱问题的应用范围涵盖了航空货运、海运物流、仓储管理和工业制造等多个领域, 该问题的研究和应用可大幅度提高运输效率和降低成本, 具有重要的理论和应用价值。

三维装箱问题求解方法主要包括精确算法、启发式算法以及元启发式算法三大类。精确算法包括分枝定界^[3]、整数规划^[4]和列生成^[5]等, 精确算法虽然在理论上能获得最优解, 但只适合求解装载规模较小的三维装箱问题。启发式算法是一类利用经验性规则来寻找问题解的算法, 包括分层法^[6]、堆塔法^[7]、极点法^[8]和筑块法^[9]等, 启发式算法具有可解释性强和求解速度快的优势, 但其求解质量不高。元启发式算法可以在有限的计算资源下寻求近似最优解, 包括禁忌搜索算法^[10]、遗传算法^[11]、模拟退火算法^[12]和贪婪随机自适应搜索算法^[13]等, 这类算法可兼顾求解效率与解的质量, 已被广泛应用于三维装箱问题求解。贪婪随机自适应搜索算法 (Greedy Randomized Adaptive Search Procedure, GRASP) 和路径重连算法 (Path-Relinking, PR) 是两个被广泛采用的智能优化技术, GRASP 是基于多起点搜索的元启发式算法, 其核心思想是对“构造—改良”策略的概率化重复, 每次迭代先以自适应贪心和随机化的方式构造一个初始可行解, 随后以局部搜索的方式对初始解进行改进, 在组合优化领域得到广泛应用^[14]。而 PR 是一种高效的局部搜索策略, 以两个已知的解为起点和终点, 将起点解通过一系列有效操作逐步向终点解靠近, 在此过程中生成一系列中间解, 从中发掘潜在的改进解。

启发式与元启发式算法依赖大量针对特定问题的规则构建与参数调优, 具有人工依赖性强与泛化能力不足的问题, 为进一步提升求解问题的灵活性, 近年来研究者开始采用超启发式算法

(Hyper-heuristic algorithms) 求解组合优化问题, 超启发式算法并不直接解决问题, 它通过高层控制策略动态地选择、组合或生成低层启发式规则来求解问题, 从而实现对复杂问题的自适应求解。典型的高层控制策略包括基于遗传算法^[15]、基于模拟退火算法^[16]和基于蚁群算法^[17]等, 超启发式算法被广泛应用于运输和物流等领域, 如车间调度^[18]和选址^[19]等问题的求解和优化。Q-learning 算法最早由 Watkins 提出^[20], 是一种基于马尔可夫决策过程的无模型的时序差分方法, 目标是直接逼近最优动作价值函数, 已有学者设计了基于 Q-learning 控制策略的超启发式算法来求解码头集卡路径优化^[21]、多模式资源约束项目调度^[22]和带容量约束的车辆路径问题^[23], 取得了不错的成效。

本文构建基于“构造—学习—强化”机制的混合超启发式算法来求解三维装箱问题, 在超启发式算法的高层, 先利用 GRASP 生成受限候选表 (Restricted Candidate List, RCL), 再由 Q-learning 在 RCL 上自适应选择低层算子, 并通过 PR 对算子序列进行强化; 低层则设计了九类启发式算子来优化算子序列, 再由最深左下角填充法与三空间分割法的构造型启发式算法对算子序列进行快速解码与评估, 实现从算子序列到放置方案的高效映射。算例分析表明, 该算法表现出寻优能力强、稳定性高和泛化性好等特点, 并且算法在小规模问题中, 多数算例与最优解几乎保持一致; 大规模算例中, 算法平均装载率稳定, 收敛速度快, 明显优于对比算法。在两组企业实例上, 本文算法也同样表现优异, 与其他启发式算法相比能有效提升货物装载率, 这表明算法在三维装箱问题求解中可稳定获得高质量解, 并且算法具备良好的可迁移性, 可广泛应用于求解其他类似的组合优化问题。

2 三维装箱数学模型

设有一类尺寸固定、数量不限的箱子, 其长、宽、高分别为 L 、 W 、 H , 以及一组包含 n 件待装货物集合 $I = \{1, 2, 3, \dots, n\}$, 待装货物 $i (i \in I)$ 对应的长、宽、高分别为 l_i, w_i, h_i 。在不违反空间重叠、货物不超出边界等约束条件的前提下, 将这 n 件货物合理放置于尽可能少的箱子中。该问题假设如下:

- (1) 箱子及货物的大小尺寸固定, 且为规则长方体。
- (2) 箱子以及货物都是刚性物体, 不能被压缩和变形。
- (3) 货物允许旋转后放置, 但货物放置后需与箱子对应的面平行。
- (4) 货物在箱子中不能相互重叠或干涉, 并且货物不能超过箱子的边界范围。

相关参数与决策变量的具体含义如表 1 所示。

表 1 相关参数与决策变量

类型	符号	含义
集合	I	货物集合, $i, k \in I = \{1, 2, \dots, n\}$
	R	货物的装载方向集合, $r \in R = \{1, 2, \dots, 6\}$
	D	相对位置方向集合, $d \in D = \{1, 2, \dots, 6\}$
参数	n	待装货物数量

	L, W, H	箱子的长、宽和高
	l_i, w_i, h_i	货物 i 的原始长、宽和高
	M	足够大的正数
	x_i, y_i, z_i	货物 i 的左后下角在箱子内的坐标
	p_i	货物 i 是否放入箱子
决策变量	o_i^r	货物 i 是否采用第 r 种装载方向, 6种装载方向如图1所示
	b_{ik}^d	判断货物 i 和 k 的方向位置

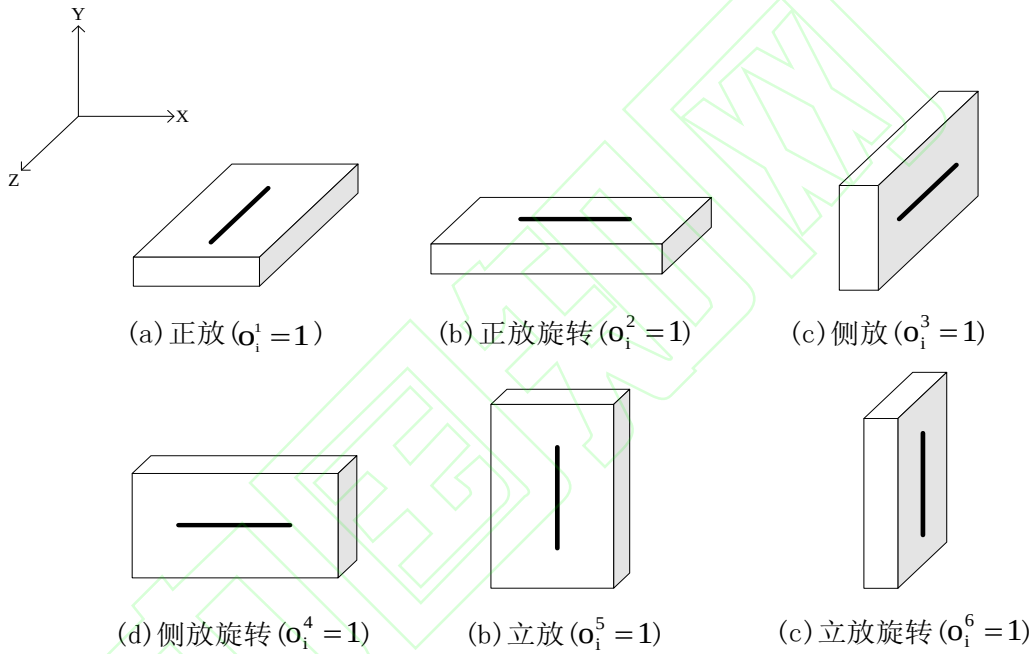


图1 货物装载方向示意图

三维装箱问题模型如公式(1)至(14)

$$Z = \max \frac{\sum_{i=1}^n l_i w_i h_i p_i}{LWH} \quad (1)$$

$$x_i + l_i(o_i^2 + o_i^4) + w_i(o_i^4 + o_i^2) + h_i(o_i^5 + o_i^6) \leq L + (1 - p_i)M \quad \forall i \quad (2)$$

$$y_i + w_i(o_i^2 + o_i^3) + h_i(o_i^4 + o_i^2) + l_i(o_i^4 + o_i^6) \leq W + (1 - p_i)M \quad \forall i \quad (3)$$

$$z_i + h_i(o_i^2 + o_i^3) + l_i(o_i^4 + o_i^6) + w_i(o_i^4 + o_i^3) \leq H + (1 - p_i)M \quad \forall i \quad (4)$$

$$\sum_{i=1}^6 o_i^r = p_i \quad \forall t \quad (5)$$

$$x_i + l_i(o_i^2 + o_i^4) + w_i(o_i^1 + o_i^3) + h_i(o_i^5 + o_i^6) \leq x_k + (1 - b_{ik}^1)M \quad \forall i \neq k \quad (6)$$

$$x_k + l_k(o_k^2 + o_k^4) + w_k(o_k^1 + o_k^3) + h_k(o_k^5 + o_k^6) \leq x_i + (1 - b_{ik}^2)M \quad \forall i \neq k \quad (7)$$

$$y_i + w_i(o_i^2 + o_i^6) + h_i(o_i^1 + o_i^3) + l_i(o_i^4 + o_i^5) \leq y_k + (1 - b_{ik}^3)M \quad \forall i \neq k \quad (8)$$

$$y_k + w_k(o_k^2 + o_k^6) + h_k(o_k^1 + o_k^3) + l_k(o_k^4 + o_k^5) \leq y_i + (1 - b_{ik}^4)M \quad \forall i \neq k \quad (9)$$

$$z_i + h_i(o_i^2 + o_i^3) + l_i(o_i^1 + o_i^6) + w_i(o_i^4 + o_i^5) \leq z_k + (1 - b_{ik}^5)M \quad \forall i \neq k \quad (10)$$

$$z_k + h_k(o_k^2 + o_k^3) + l_k(o_k^1 + o_k^6) + w_k(o_k^4 + o_k^5) \leq z_i + (1 - b_{ik}^6)M \quad \forall i \neq k \quad (11)$$

$$\sum_{d=1}^6 b_{ik}^d \geq p_i + p_k - 1 \quad \forall i \neq k \quad (12)$$

$$x_i, y_i, z_i \geq 0 \quad \forall i \quad (13)$$

$$p_i, b_{ik}^d, o_i^r \in \{0,1\} \quad \forall i, k, r, d \quad (14)$$

公式(1)为目标函数，表示最大化箱子装载率；公式(2)-(4)确保货物不能超出箱子边界；公式(5)确保货物最多采用一个方向装载；公式(6)-(12)保证位于同一箱子中的任意两货物在空间上没有重叠；公式(13)-(14)为决策变量取值范围。

3 构造型启发式算法

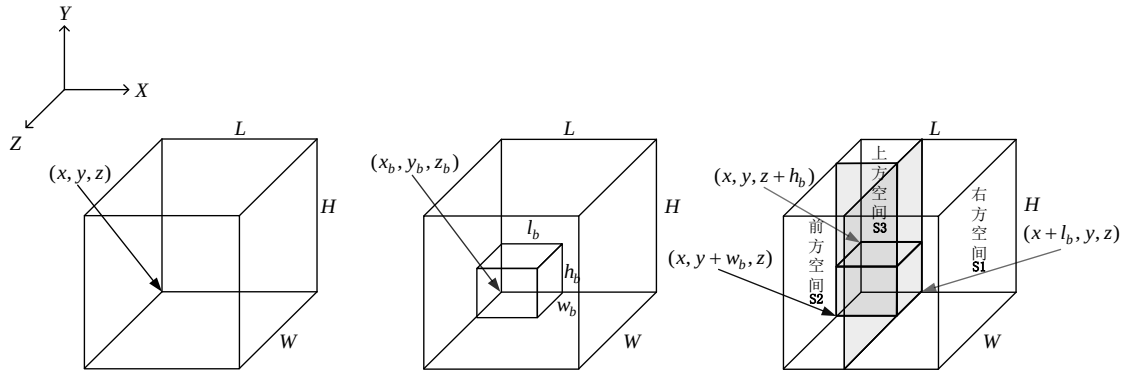
构造型启发式算法以预设的规则或启发式准则，逐一将货物装载至箱子之中。本文采用的最深左下角填充法与三空间分割法来完成货物的装载过程。

3.1 最深左下角填充法

最深左下角填充法的核心思想是在装载货物过程中，将货物优先放置最靠近箱子后壁的位置，即按照 z 坐标最大（最深）优先，其次 y 坐标最小（最下），最后 x 坐标最小（最左）的顺序为待装货物选择最终放置点。

3.2 三空间分割法

空间分割是三空间分割法的核心环节之一，其基本思想是在每次货物放置后，依据新放入货物的位置与尺寸，将被装载的空间分割为三个子空间。设当前可用空间块 $S = (x, y, z, L, W, H)$ ，其中 (x, y, z) 为空间块的起始坐标， (L, W, H) 为空间块的尺寸；货物 $B = (x_b, y_b, z_b, l_b, w_b, h_b)$ ，其中 (x_b, y_b, z_b) 为其左后下角在箱子中的坐标， (l_b, w_b, h_b) 为箱子的尺寸。当 B 与 S 存在重叠时， S 被分割为三个相互独立的新空间：若 $L > l_b$ ，则在货物右侧生成空间 S_1 ；若 $W > w_b$ ，则在货物前方生成空间 S_2 ；若 $H > h_b$ ，则在货物上方生成空间 S_3 ，如图2所示。



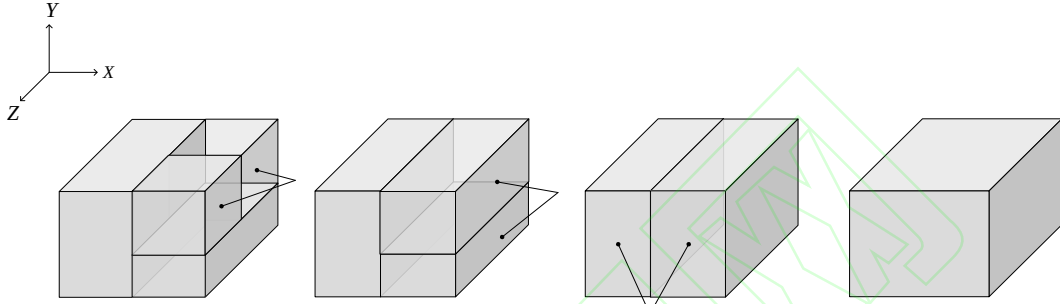
(a) 初始空间 S

(b) 放入箱体 B

(c) 三空间分割

图 2 三空间分割法

随着货物的不断被装入，箱体空间分割将不断生成新的碎片空间。为减少空间碎片、提升装载效率，需要对空间进行合并。空间合并的基本思想是：对于任意两块未被占用的空间块，若其在除某一维度以外的其余两个维度上位置与尺寸完全一致，且在该维度上相邻，则将其合并为一块更大的空间，一般有宽度方向、高度方向和长度方向三种合并方式，如图 3 所示。



(a) 宽度方向合并

(b) 高度方向合并

(c) 长度方向合并

(d) 合并后空间

图 3 空间合并示意图

3.3 算法伪代码

基于最深左下角和三空间分割的构造型启发式算法实现的伪代码如算法 1 所示：

算法 1 三维装箱构造型启发式算法

算法 1: DBLFBased3DLoad

输入：箱子尺寸 (L, W, H) ; 装箱序列 I 以及对应尺寸 (l_i, w_i, h_i) ; 装载方向集合 R

初始化：空闲空间集合 $S \leftarrow \{(0,0,0, L, W, H)\}$; 放置结果 $Placed \leftarrow \emptyset$

输出：放置方案 $Placed$; 装载率 $NewVal$

```

1  for each  $i \in I$  do
2      sort( $S$ ) by ( $z \uparrow, y \uparrow, x \uparrow$ )
3       $Placed \leftarrow false$ 
4      for each  $s \in S$  do
5          for each  $r \in R$  do
6               $(l_i^r, w_i^r, h_i^r) \leftarrow$  由 $(l_i, w_i, h_i)$ 按装载方向 $r$ 置换得到
7              if  $(l_i^r \leq s.l) \wedge (w_i^r \leq s.w) \wedge (h_i^r \leq s.h)$  then
8                   $x_i \leftarrow s.x$ ;  $y_i \leftarrow s.y$ ;  $z_i \leftarrow s.z$ 
9                   $\odot o_i^r \leftarrow 1, o_i^{r'} \leftarrow 0 \quad (\forall r' \neq r)$ 
10                  $p_i \leftarrow 1$ 
11                  $Placed \leftarrow Placed \cup (i, x_i, y_i, z_i, r)$ 

```

```

12           $S \leftarrow S \setminus \{s\}$ 
13           $S_{new} \leftarrow Split(s, (l_i^r, w_i^r, h_i^r))$ 
14           $S \leftarrow S \cup S_{new}$ 
15           $S \leftarrow Cleanup(S)$ 
16           $S \leftarrow MergeAll(S)$ 
17           $NewVal \leftarrow PackingRate(Placed)$ 
18          break
19        end if
20      end for
21    if Placed then break
22  end for
23  if pi = 1 then continue
24  end for
25 return Placed, NewVal

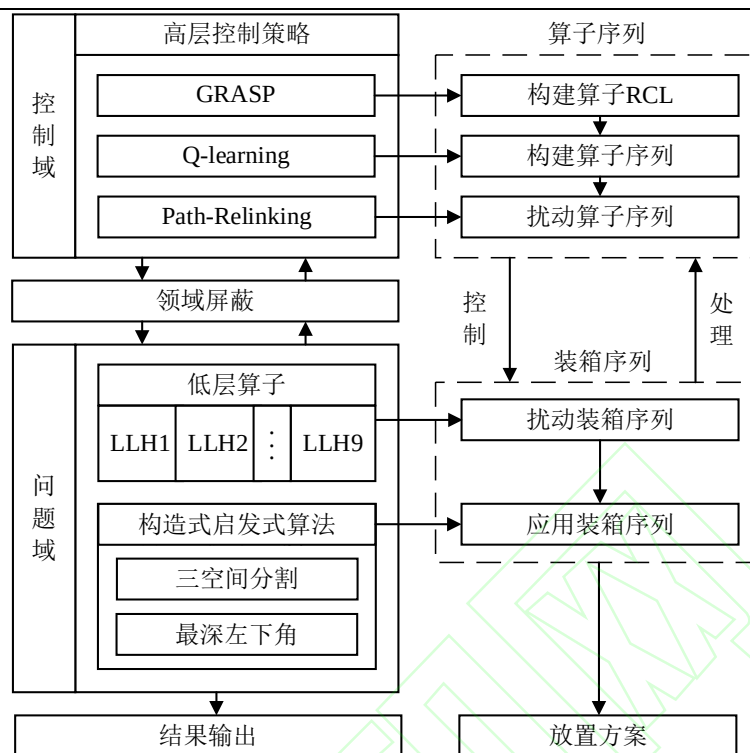
```

算法 1 的 2-7 行将空闲空间集合 S 按 z （最深）、 y （最下）、 x （最左）的顺序排序，随后依次枚举每个空间 s 与六种正交装载方向 r ，并据旋转后的尺寸 (l_i^r, w_i^r, h_i^r) 判断货物 i 能否装入空间 s ，若可行，第 8-12 行则把货物放置在该空间的左后下角并设置放置坐标与装载方向指示变量，将空间 s 从集合中移除。第 13 行将空间 s 切割生成新的子空间。第 14-16 行把新的子空间并入空闲空间集合，随后分别调用 *Cleanup* 和 *MergeAll* 方法对空闲空间进行清理和合并。第 17-18 行使用 *PackingRate* 方法计算当前装载率。第 21-23 行放置下一件货物。当所有货物处理完毕后返回放置方案与装载率，算法结束。

4 超启发式算法

4.1 总体框架

算法由高层控制域与低层问题域两部分组成。高层控制域包括 GRASP、Q-learning 和 PR 三个组件。问题域层面则包含 9 种低层算子和实际装箱操作的构造型启发式算法，他们分别将高层选择的算子序列转化为具体的装箱序列并最终完成货物的放置。整个算法框架如图 4 所示。



4.2 核心机制

GRASP、Q-learning 和 PR 这三个组件构成了“构造—学习—强化”控制机制，其本质上是一个内嵌学习能力的协同优化框架。GRASP 是一种基于多起点的搜索算法，其搜索本质上是对多种低层算子（low-level heuristics, LLH）进行自适应性采样，得到一组高质量的底层算子数据集，该数据集为 Q-learning 算法智能决策提供了丰富的训练样本，使后者逐步学习得到问题状态和低层算子之间的映射关系，保障了算法整体的自适应性及其对优化目标的持续逼近。而 PR 则为 Q-learning 所引导的算子序列提供精细调优与局部强化，则进一步提高局部改进效率并增强搜索稳定性，即 GRASP 保证探索的广度，Q-learning 提供长期收益导向，PR 能够有效调优并逃离局部最优。

4.3 适应性分析

本文提出的超启发式算法的高层控制机制融合了 GRASP、Q-learning 与 PR，通过对底层算子进行动态选择与组合来引导搜索。由于本算法的搜索过程是对底层算子执行顺序的扰动，从而实现在解的序列空间中进行探索与优化。因而该算法框架适用于基于选择的超启发式算法并且解可表示为序列的组合优化问题，例如任务调度、路径规划或装载排序等场景——只要问题的解能抽象为某种顺序（如任务执行顺序、节点访问顺序、物品装载顺序）并能够基于该序列构造解，即可在本框架的序列空间内进行优化。但该框架仍存在一定适用边界：若问题虽然可以以序列表示，但目标函数对序列扰动并不敏感，且主要改进依赖全局重构而非局部调整，则基于序列扰动的搜索难以持续获

得有效优化；若约束强度过高，使得扰动后频繁产生不可行解并且修复成本高，则该框架下运行效率将受到影响；此外，若算子库规模过小、算子类型同质性较强，可组合的优化策略本身受限，该框架也可能失效。因此该算法特别适用于货物类型多样、尺寸分布相对稳定的批量装箱场景，此外还要求精心设计一定数量的、并具备功能互补性的低层算子集合，确保能够生成多样化的装箱模式。而在序列极端、基础数据呈现显著分布差异等情况下，则需针对性地设计算子或进行框架扩展。

4.4 高层策略

4.4.1 基于 GRASP 的构造算法

该算法利用 GRASP 算法来选择低层算子，构造生成 RCL，伪代码如算法 2 所示。

算法 2 基于 GRASP 的构造算法伪代码

算法 2: GRASP-HHS

输入： 入选 RCL 比例 α ；算子集合 $A = \{a_1, a_2, \dots, a_n\}$ ；装箱序列 I 以及对应尺寸 (l_i, w_i, h_i)

初始化： $RCL \leftarrow \emptyset$ ；候选集 $C \leftarrow \emptyset$

输出： RCL 列表

```

1  for each  $a \in A$  do
2       $I \leftarrow \text{apply 算子 } a$ 
3       $(Placed, NewVal) \leftarrow DBLFBased3DLoad$ 
4       $C \leftarrow C \cup \{ (a, Placed, NewVal, I) \}$ 
5  end for
6   $k \leftarrow \text{round}(\alpha \cdot |A|)$ 
7  sort  $C$  by rate desc
8   $RCL \leftarrow \text{first } k \text{ elements of } C$ 
9  return  $RCL$ 

```

算法 2 的 1-4 行使用低层算子 $a \in A$ 在装箱序列 I 上进行扰动，然后调用算法 *DBLFBased3DLoad* 评估得到放置方案与装载率 $(Placed, NewVal)$ ，并把对应的算子、放置方案、装载率和扰动后装箱序列放入候选集 C 中。第 6-8 行按比例参数 α 计算 RCL 大小，将候选集 C 按装载率降序排序后，取前 k 个组成 RCL，以保证每次扰动都在局部最优算子中进行选择。第 9 行返回 RCL，算法结束。

4.4.2 基于 Q-learning 的学习算法

在算子的最终选择阶段，即在 GRASP 算法生成 RCL 后，引入 Q-learning 学习机制。对于每个 RCL 中的算子，依据当前状态在 Q 表中检索其历史表现，以概率 ε 在 RCL 中随机选择一个算子，实现探索多样性；以概率 $1 - \varepsilon$ 选择 RCL 中 Q 值最高的算子，利用以往搜索经验进行学习实现自适应决策。若 RCL 中算子的 Q 值相同，则随机在 RCL 中选择算子。

1.状态空间设计：状态 s 以当前解的装载率为核心度量，为了减少状态数目并提高学习效率，将装载率按状态离散粒度 t 分割为若干离散区间，每个区间作为一个状态 s_t ，在0% ~ 100%的装载率范围内，共可得到 $100/t$ 个离散状态，从而有效降低状态维度并提升 Q-learning 的学习效率。

2.动作空间：动作 a 即为低层扰动算子的选择。每种低层算子都作为动作空间 a 的一个元素，动作空间 $A = \{a_1, a_2, \dots, a_n\}$ ， n 为算子总数。

3.Q 表：Q 表采用嵌套字典表示： $Q_{table}: s \rightarrow \{a \rightarrow Q(s, a)\}$ 。其中外层键是状态 s ，由装载率以约 2%步长离散化得到的整数；内层是动作到 Q 值的映射，在首次访问某状态时按所有候选动作将 Q 值初始化为 0。

4.奖励函数：奖励 r 定义为应用算子 a 后装载率的提升量，即： $r_t = \lambda(f(new) - f(current))$

其中， λ 为奖励系数， $current$ 和 new 分别表示扰动前后解的装载率。若是无提升，则给予小负奖励，避免算法长期停留在原地陷入局部最优解。

5.Q 表更新规则：

获得即时奖励后，算法根据贝尔曼方程对 Q 表进行更新：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \beta[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)] \quad (15)$$

其中 β 为学习率， γ 为折扣因子， s_{t+1} 为扰动后的新状态。

这里的 Q-learning 采用近似学习策略，首先，在状态空间方面，通过引入状态离散粒度参数 t ，将连续装载过程划分为有限状态。其次，在动作空间方面，高层决策的动作被定义为从固定的低层算子集合中选择算子，其规模在实验过程中保持不变，因此动作空间有限。最后，在探索与更新策略上，采用固定探索概率 ϵ 以及固定学习率 β ，并在预设的最大迭代次数下终止算法运行。该过程并不依赖于价值函数在理论意义上的渐进最优性，而是通过构造机制维持必要的多样性、学习机制引导搜索方向以及强化机制的局部调优，在有限计算开销下形成稳定算子调度偏好，确保算法的寻优能力和收敛性。

算法伪代码如算法 3 所示：

算法 3 基于 Q-learning 的学习算法伪代码

算法 3: QL-HHS

输入： Q 表 $Q(s, a)$ ；装箱序列 I 以及对应尺寸 (l_i, w_i, h_i) ；

输出： 选择的算子 a_t ；更新后的 Q 表

```

1  RCL ← GRASP-HHS
2  NewVal ← DBLFBased3DLoad
3  if rand( ) ≥ ε then
4     $a_t \leftarrow \operatorname{argmax}\{a \in RCL\} Q(s_t, a)$ 
5  else
6     $a_t \leftarrow \operatorname{Random}(RCL)$ 
```

```

7  end if
8   $I \leftarrow \text{apply 算子 } a_t$ 
9   $\text{NewVal}_{a_t} \leftarrow \text{DBLFBased3DLoad}$ 
10  $\Delta \leftarrow \text{NewVal}_{a_t} - \text{NewVal}$ 
11  $r_t \leftarrow \lambda \cdot \Delta$ 
12  $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \beta[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$ 
13 return  $Q, a_t$ 

```

算法 3 的 1-2 行由算法 *GRASP-HHS* 生成 RCL，调用装箱过程使用的算法 *DBLFBased3DLoad*，得到当前装载率 *NewVal* 并作为基线。第 3-7 行以 $1 - \varepsilon$ 的概率，选择使 $Q(s_t, a_t)$ 最大的动作，否则通过 *Random* 方法随机选取 RCL 中的动作。第 8-12 行在原装箱序列 *I* 上执行所选动作，通过算法 *DBLFBased3DLoad* 获得新装载率 NewVal_{a_t} ，计算改进幅度并给出即时奖励，最后更新 Q 表。最终返回更新后的 Q 表和被选择算子，算法结束。

4.4.3 基于 Path-Relinking 的强化算法

该算法主要将 PR 算法用于高层选择策略结果的强化，使用 PR 对算子序列进行扰动，从而进一步实现算子序列的优化。PR 选取历史全局最优算子序列作为起点，将当前迭代得到的算子序列作为终点，逐步修正算子序列的排列，使从优秀算子序列逐渐向当前算子序列移动，直到与当前算子序列完全一致。在每一步转换过程中，对新生成的解进行装箱仿真与评估，并保留路径上最优的解作为候选。最终，若路径中存在优于当前解的方案，则更新全局最优解及相关参数。伪代码如算法 4 所示：

算法 4 基于 Path-Relinking 的强化算法伪代码

算法 4: PR-HHS

输入： 起点算子序列 Seq_{start} ；终点算子序列 Seq_{target}

输出： 最优放置方案 *BestPlaced*；最优装载率 *BestVal*；最优算子序列 *BestSeq*；最优装箱序列 *BestI*

```

1  while  $\text{Seq}_{start} \neq \text{Seq}_{target}$  do
2       $D \leftarrow \{ j \mid \text{Seq}_{start}[j] \neq \text{Seq}_{target}[j], j = 1..L \}$ 
3      choose  $j \in D$ 
4       $\text{Seq}_{start}[j] \leftarrow \text{Seq}_{target}[j]$ 
5      for  $k = 1..L$  do
6           $a \leftarrow \text{Seq}_{start}[k]$ 
7           $I \leftarrow \text{apply 算子 } a$ 
8           $(\text{Placed}, \text{NewVal}) \leftarrow \text{DBLFBased3DLoad}$ 

```

```

9      if NewVal > BestVal then
10         BestVal ← NewVal
11         BestPlaced ← Placed
12         BestSeq ← Seqstart
13         BestI ← I
14     end if
15 end for
16 end while
17 return BestPlaced, BestVal, BestSeq, BestI

```

算法 4 的第 1-3 行表示在起点序列 Seq_{start} 与终点序列 Seq_{target} 存在不一致时进行迭代, 先将两个序列不一致的位置存入集合 D 中, 再从集合 D 中取出第一个位置信息, 并用 Seq_{target} 该位置的算子替换 Seq_{start} 对应算子。第 5-8 行将替换后的完整算子序列依次作用在当前装箱序列上, 期间调用算法 *DBLFBased3DLoad* 评估得到放置方案 $Placed$ 与装载率 $NewVal$; 若 $NewVal$ 优于当前最优, 第 9-14 行则同步更新最优装载率 $BestVal$ 、最优放置方案 $BestPlaced$ 、最优算子序列 $BestSeq$ 和最优装箱序列 $BestI$ 。最后当起点序列被逐步同化直至与目标序列一致时终止, 并返回 $BestPlaced, BestVal, BestSeq, BestI$, 算法结束。

4.5 低层算子

低层算子直接影响到解空间的探索广度与算法的局部搜索能力, 考虑到装箱序列是对最终装载率起决定性的重要因素, 本文对装箱序列进行扰动以获得更好的装箱结果, 据此共设计实现了九种低层算子。具体如下:

LLH1: 两点逆序, 随机选择序列中两个位置 i, j , 将 i 和 j 之间的所有元素逆序排列, 如图 5 所示。



图 5 LLH1 示意图

LLH2: 两点交换, 随机选择序列中两个位置 i, j , 交换其位置, 如图 6 所示。

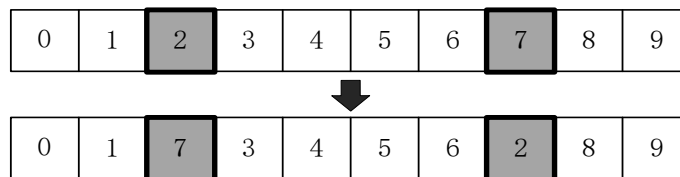


图 6 LLH2 示意图

LLH3: 三点扰动, 随机选取三个切割点 $i < j < k$, 将序列划分为四段, 并将两段区块 $[i:j]$ 和 $[j:k]$ 分别逆序重组, 其他部分顺序保持不变, 如图 7 所示。

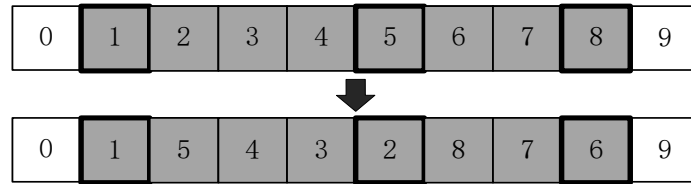


图 7 LLH3 示意图

LLH4: 区块插入, 随机选取一段子序列, 移除并插入到序列其他随机位置, 如图 8 所示。

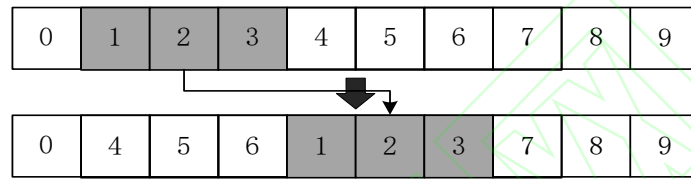


图 8 LLH4 示意图

LLH5: 相邻交换, 随即交换序列中一对相邻的元素, 如图 9 所示。

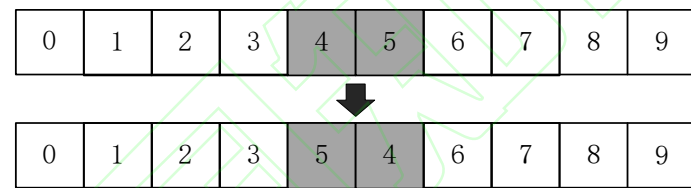


图 9 LLH5 示意图

LLH6: 区块交换, 随机选取两段不重叠区块, 互换二者在序列中的位置, 如图 10 所示。



图 10 LLH6 示意图

LLH7: 逆序插入, 随机选择一段区块, 先将其逆序, 然后插入到新位置, 如图 11 所示。

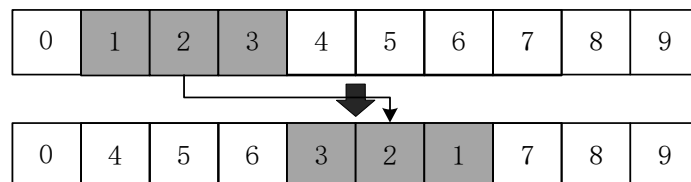


图 11 LLH7 示意图

LLH8: 分段随机打乱, 随机选取解序列中的一个区块, 将该区块内部所有元素进行完全随机打乱

乱，其余部分顺序保持不变，如图 12 所示。

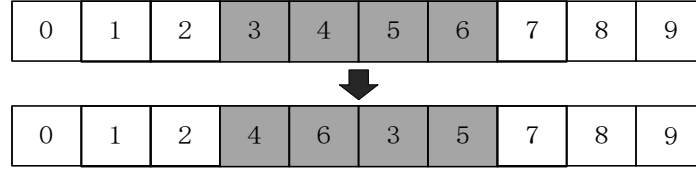


图 12 LLH8 示意图

LLH9: 随机打乱，将该序列内部所有元素进行完全随机打乱，如图 13 所示。

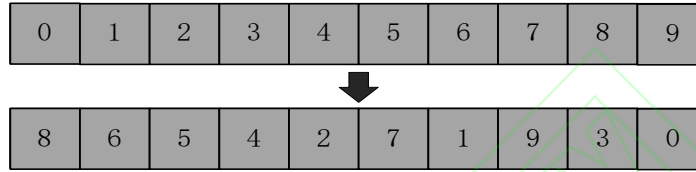


图 13 LLH9 示意图

4.6 整体算法流程

整体算法框架根据其功能共划分为四个功能模块，分别为高层 *GRASP-HHS*（算法 2）、*QL-HHS*（算法 3）和 *PR-HHS*（算法 4）以及低层构造启发式 *DBLFBased3DLoad*（算法 1），高层的算法在运行过程调用低层算法，如图 14 所示。

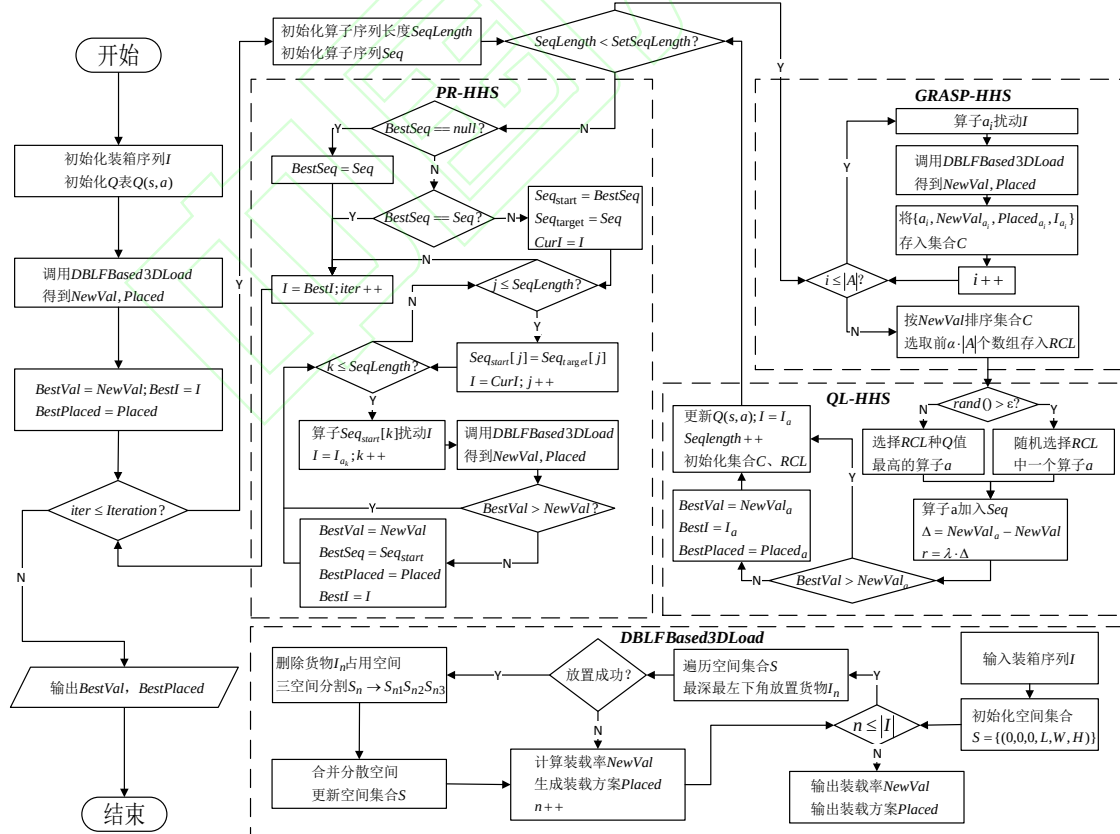


图 14 整体算法框架图

首先,按货物体积降序排列生成初始装箱序列 I ,初始化 Q 表。随后进入迭代,在每一次迭代前将算子序列长度 $SeqLength$ 以及算子序列 Seq 初始化,若当前算子序列长度小于设定值,则调用算法 $GRASP-HHS$ 遍历算子集,对每个候选算子通过算法 $DBLFBased3DLoad$ 评估并记录其对应值,再按比例生成受限候选表 RCL 。接着由算法 $QL-HHS$ 在 RCL 上选择算子,依据即时反馈进行奖惩与 Q 值更新,并对当前的装载率进行赋值以及算子序列和当前序列长度的更新。如果当前的装载率大于最好的装载率则对最优放置方案和最优装载率进行赋值。如果算子序列满足设定长度并且最优算子序列仍为空,则将当前算子序列赋值给最优算子序列;若当前算子序列长度满足设定值,则调用算法 $PR-HHS$ 以历史最优序列与当前序列为两端,按逐位替换产生路径,并反复用算法 $DBLFBased3DLoad$ 评估中间序列,取路径上最优解并记录。若迭代次数未满足要求,则用最优装箱序列 $BestI$ 作为初始解进行下一轮迭代;否则,输出全局的最优放置方案 $BestPlaced$ 、最优装载率 $BestVal$,算法结束。整体算法伪代码如算法5所示:

算法 5 整体算法伪代码

算法 5 Hy-HH for 3D-BPP

输入: 最大迭代次数 $Itertion$; 最大算子序列长度 $SetSeqlength$

初始化: 装箱序列 $I \leftarrow sort(i)$; Q 表 $\leftarrow Q(s,a)$; 算子序列长度 $SeqLength \leftarrow 0$; 算子序列 $Seq \leftarrow []$

输出: 最优放置方案 $BestPlaced$; 最优装载率 $BestVal$

```

1  while iter < Itertion do
2      if SeqLength < SetSeqLength then
3          RCL ← apply GRASP-HHS
4          (Q, at) ← apply QL-HHS
5          (Placed, NewVal, I) ← RCL[at]
6          SeqLength ← SeqLength + 1
7          Append(Seq, at)
8          if NewVal > BestVal then
9              BestPlaced ← placed
10             BestVal ← NewVal
11             BestI ← I
12         end if
13     if SeqLength = SetSeqLength and BestSeq = None then
14         BestSeq ← Seq
15     end if
16     else

```

```

17      Seqstart = BestSeq
18      Seqtarget = Seq
19      (BestPlaced, BestVal, BestSeq, BestI) ← Apply PR-HHS
20      end if
21      I = BestI
22      Iter ← Iter + 1
23  end while
24  return BestPlaced, BestVal

```

4.7 算法时间复杂度分析

整体计算过程可划分为低层算子执行阶段与高层控制与学习阶段两部分。其中，低层算子负责最终装箱方案的构造与调整，是主要的计算开销来源；高层算法用于算子选择与策略更新，其计算成本相对较低。算法时间复杂度涉及的相关符号包括：货物数量 n ，算子数量 A ，算子序列长度 L ，迭代次数 I 。

4.7.1 低层算法复杂度

算法 1 在给定装箱顺序的前提下生成可行的放置方案，其复杂度源自对剩余货物的尝试过程。最坏情形为：第一个空间对所有 n 件货物且尝试 6 个方向旋转后仍放置失败；第二个空间对剩余的 $n - 1$ 件货物以及尝试全部方向仍放置失败，以此类推算法 1 的时间复杂度为 $T_1(n)$ 为 $T_1(n) = O(6[n + (n - 1) + \dots + 1]) = O(3n^2 + 3n)$ 。

4.7.2 高层算法复杂度

算法 2 首先使用算子 a 对长度为 n 的装箱序列执行一次扰动，其时间复杂度为 $O(n)$ ，随后调用算法 1 进行装载评估，因此算法 2 的时间复杂度 $T_2(n)$ 为： $T_2(n) = O(A(T_1(n) + n)) = O(A(3n^2 + 4n))$ 。算法 3 使用装载率差值来更新 Q 表，其时间复杂度来源两次调用算法 1，因此算法 3 的时间复杂度 $T_3(n)$ 为： $T_3(n) = O(2T_1(n)) = O(6n^2 + 6n)$ 。算法 4 最坏情况为起点序列和终点序列完全不一样，则需要替换 L 次位置，并每次替换都要调用算法 1 遍历算子序列，其时间复杂度 $T_4(n)$ 为： $T_4(n) = O(L \cdot L \cdot T_1(n)) = O(L^2(3n^2 + 3n))$ 。

4.7.3 整体算法复杂度

整体算法每一轮迭代构造一条长度为 L 的算子序列，需要调用算法 2 和算法 3 共 L 次，序列构造完成后调用算法 4 执行一次，因此在满足 I 次迭代下，整体算法的时间复杂度 $T_{total}(n) = O(I(LT_2(n) + T_3(n)) + T_4(n)) = O(I((3LA + 6L + 3L^2)n^2 + (4LA + 6L + 3L^2)n))$ 。随货物规模 n 的逐渐增大， $T_{total}(n)$ 可以进一步简化为： $T_{total}(n) = O(I \cdot L(A + L)n^2)$ 。由此可知，算法整体计算开销由算法 1 主导，外层参数 L 、 I 、 A 通过影响算法 1 的调用次数来改变算法的总耗时。当固定外层参数 L 、 I 、 A 时，算法复杂度仅由货物数量 n 主导，时间复杂度随货物数量 n 呈二次增长，可计算为 $T(n) = O(n^2)$ ，

说明算法能够以可控的时间复杂度处理规模增长问题。

5 仿真实验

采用 Python 语言实现，实验环境为：Intel(R) Core(TM) i7-11800H @ 2.30GHz 处理器，16GB 内存，求解器采用杉树科技 COPT 7.2.11。

5.1 算例简介

选用 Martello 提出的随机生成的数据集^[24]以及 Loh&Nee 所提供的三维装箱算例集^[25](下文简称为 LN)这两种算例作为实验对象。

5.1.1 Martello 数据集

Martello 定义了五种货物生成类型，并按比例生成八组测试数据集，其中货物生成类型如表 2 所示。

类型	W	L	H
	选择区间		
type 1	$(1, \frac{W}{2}]$	$(\frac{2L}{3}, L]$	$(\frac{2H}{3}, H]$
type 2	$(\frac{2W}{3}, W]$	$(\frac{2L}{3}, L]$	$(1, \frac{H}{2}]$
type 3	$(\frac{2W}{3}, W]$	$(1, \frac{L}{2}]$	$(\frac{2H}{3}, H]$
type 4	$(\frac{W}{2}, W]$	$(\frac{L}{2}, L]$	$(\frac{H}{2}, H]$
type 5	$(1, \frac{W}{2}]$	$(\frac{L}{2}, L]$	$(\frac{H}{2}, H]$

按不同货物生成类型比例分别生成八组测试数据集 $class \in \{1, 2, 3, 4, 5, 6, 7, 8\}$ ，当 $class \in \{1, 2, 3, 4, 5, 8\}$ 时，箱子的尺寸 $L = H = W = 100$ ；当 $class = 6$ 时， $L = H = W = 10$ ；当 $class = 7$ 时， $L = H = W = 40$ 。具体生成方式如下：

当 $class = 1$ 时，取 60% 的概率生成 $type = 1$ 的货物类，其余 40% 的概率平均分给 $type \in \{2, 3, 4, 5\}$ 。
 当 $class = 2$ 时，取 60% 的概率生成 $type = 2$ 的货物类，其余 40% 的概率平均分给 $type \in \{1, 3, 4, 5\}$ 。
 当 $class = 3$ 时，取 60% 的概率生成 $type = 3$ 的货物类，其余 40% 的概率平均分给 $type \in \{1, 2, 4, 5\}$ 。
 当 $class = 4$ 时，取 60% 的概率生成 $type = 4$ 的货物类，其余 40% 的概率平均分给 $type \in \{1, 2, 3, 5\}$ 。
 当 $class = 5$ 时，取 60% 的概率生成 $type = 5$ 的货物类，其余 40% 的概率平均分给 $type \in \{1, 2, 3, 4\}$ 。
 当 $class \in \{6, 7, 8\}$ 时，不再通过 $type$ 来生成货物种类，而是分别在 $[1, 10]$, $[1, 35]$, $[1, 100]$ 范围内随机生成。

5.1.2 Loh&Nee 数据集

Loh&Nee 数据集提供从 LN01-LN15 共计 15 个算例，这 15 个算例在箱子尺寸、货物类型及装

载难度上均具有代表性，能够充分检验算法在不同装箱场景下的稳定性，其中除 LN02，LN06 两个算例，其他算例货物的总体积均小于箱子的容积，能被完全装入箱子中。

5.2 参数设置

为获得算法的最佳参数配置，本文采用正交试验设计对主要参数进行联合调优与敏感性分析，选取 RCL 比例参数 α 、算子序列长度 $length$ 、Q-learning 探索概率 ε 、学习率 β 、折扣因子 γ 以及状态离散粒度 t 这 6 个主要参数进行敏感性分析，每个因素设置三个水平，构成一个 3^6 的试验空间，并对每组运行 10 次取相应结果。

表 3 相关参数设置水平

	α	$length$	ε	β	γ	t
level1	0.2	4	0.1	0.1	0.85	0.1
level2	0.4	6	0.2	0.15	0.9	0.5
level3	0.6	8	0.3	0.2	0.95	1

考虑到全因子试验的计算成本，本文采用 $L27(3^{13})$ 正交表对 27 组参数组合进行试验，各因素的水平设置如表 3 所示。在此基础上，依据 $L27(3^{13})$ 正交表对 6 个因素进行编排试验，得到的 27 组参数组合及其 LN02 上的平均装载率见表 3。

表 4 $L27(3^{13})$ 正交表实验结果

编号	Level						最大装载率	平均装载率	平均运行时间
	α	$length$	ε	β	γ	t			
1	1	1	1	1	1	1	93.27	91.58	204.87
2	1	1	2	1	2	2	94.28	91.50	220.40
3	1	1	3	1	3	3	94.12	91.67	215.34
4	1	2	1	2	1	2	93.98	92.18	328.95
5	1	2	2	2	2	3	92.94	92.26	337.68
6	1	2	3	2	3	1	94.33	92.31	334.80
7	1	3	1	3	1	3	93.13	91.72	471.22
8	1	3	2	3	2	1	93.74	92.41	498.76
9	1	3	3	3	3	2	93.54	92.44	501.37
10	2	1	1	2	2	1	92.99	91.46	202.67
11	2	1	2	2	3	2	92.35	91.45	195.55
12	2	1	3	2	1	3	93.54	91.88	211.03
13	2	2	1	3	2	2	92.54	91.32	331.13
14	2	2	2	3	3	3	92.79	92.00	218.33
15	2	2	3	3	1	1	93.98	92.18	193.20
16	2	3	1	1	2	3	93.46	92.21	297.63
17	2	3	2	1	3	1	93.39	91.77	277.51

Computer Integrated Manufacturing Systems

18	2	3	3	1	1	2	92.69	91.20	291.58
19	3	1	1	3	3	1	93.58	91.31	120.80
20	3	1	2	3	1	2	93.73	92.03	120.03
21	3	1	3	3	2	3	94.64	92.12	125.45
22	3	2	1	1	3	2	93.36	91.28	205.58
23	3	2	2	1	1	3	94.53	91.92	222.55
24	3	2	3	1	2	1	92.74	91.58	252.35
25	3	3	1	2	3	3	94.00	92.05	290.30
26	3	3	2	2	1	1	92.51	91.59	281.28
27	3	3	3	2	2	2	92.24	91.40	322.85

基于表 4 中正交试验的结果,对六个三水平因素分别计算了各水平下的平均装载率,并在正交结构下对其他因素的影响进行平均,从而开展主效应分析。如图 15 结果所示,RCL 比例参数 α 随着取值的逐渐增大,其对应的平均装载率呈现出下降趋势,原因在于 RCL 比例参数 α 控制构造阶段受限候选列表的覆盖范围, α 较小时候选集中质量较高,构造解更为可靠;随着 α 增大,低质量候选被纳入并选中的概率提高,从而削弱了整体装载率。算子序列长度 $length$ 决定了单次迭代中组合搜索的深度,适中的 $length$ 有助于在质量与时间之间取得平衡,如图 16 所示,不同参数设置水平下除算子序列长度 $length$ 外,其余参数对平均运行时间的影响较小,主效应曲线较为平缓,而算子序列长度 $length$ 则随着取值增加,对运行时间的影响也越大,过大的 $length$ 会显著增加每轮计算规模,从而推高运行时间,因此在取值为 6 的时候最佳。探索概率 ϵ 影响 Q-learning 决策阶段的探索与利用权衡,中等的 ϵ 能在避免过早固化算子偏好的同时抑制过度随机搜索,因此表现出较为稳定的装载率。学习率 β 决定 Q 值更新对新回报信息的响应强度,较大的 β 会加快策略更新但可能引入波动,而较小的 β 则使更新过程更为平稳。折扣因子 γ 控制回报在时间维度上的权重分配,在以即时装载收益为主的奖励设计下,其变化对最终装载率的影响相对有限,因此主效应曲线整体较为平缓。状态离散粒度 t 用于控制状态空间划分的尺度,过于细致的划分会增加状态空间规模以及状态切换带来的策略波动,在取值为 1 的时候装载率最佳。

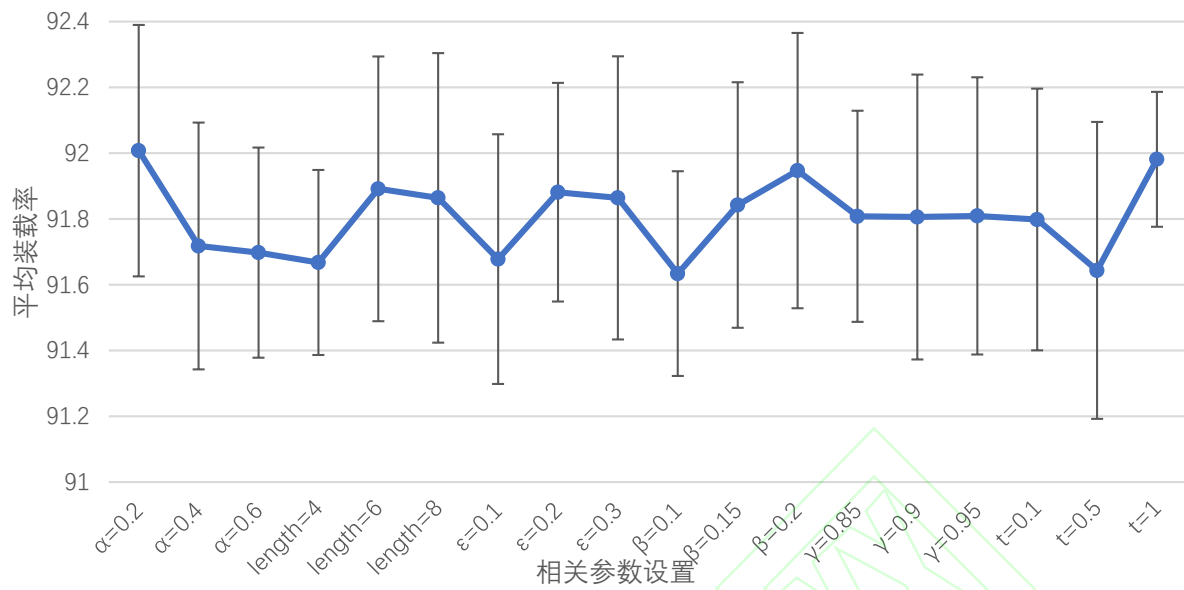


图 15 参数对装载率的主效应图

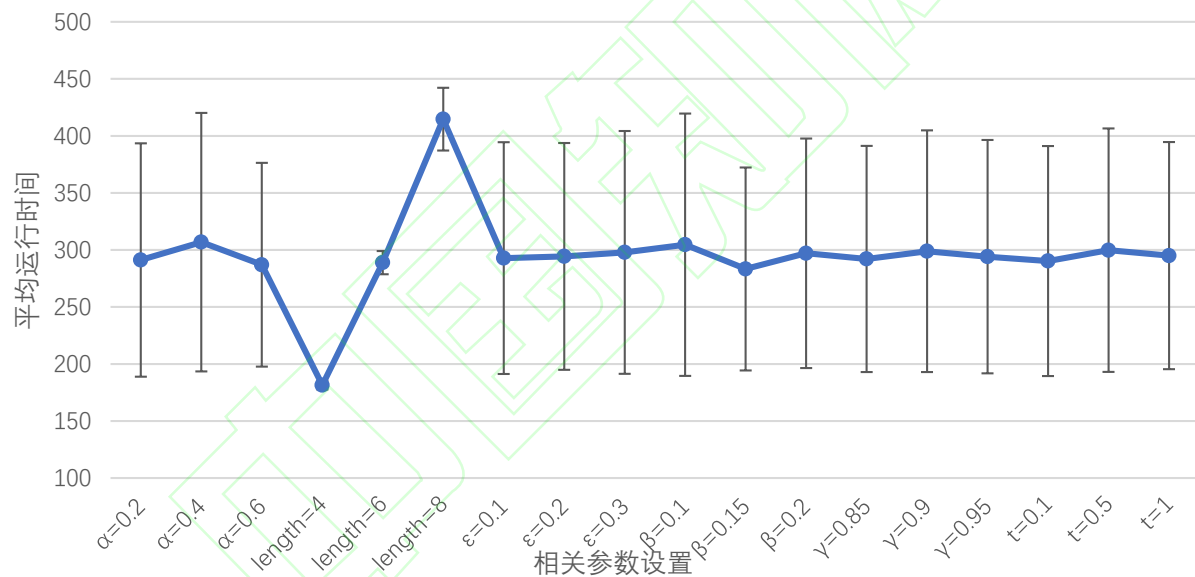


图 16 参数对运行时间的主效应图

因此综合比较下，本文选取 $\alpha = 0.2$ 、 $length = 6$ 、 $\epsilon = 0.2$ 、 $\beta = 0.2$ 、 $t = 1$ 时作为参数设置，考虑到三个水平下 γ 的影响差异较小，分别取 $\gamma = 0.85$ 、 $\gamma = 0.9$ 、 $\gamma = 0.95$ 代入以上确定的参数设置下的相同算例进行实验，结果如表 5 所示，得出 $\alpha = 0.2$ 、 $length = 6$ 、 $\epsilon = 0.2$ 、 $\beta = 0.2$ 、 $\gamma = 0.95$ 、 $t = 1$ 时为最优参数设置。

表 5 参数实验结果

α	$length$	ϵ	β	γ	t	最大装载率	平均装载率	平均运行时间
0.2	6	0.2	0.2	0.85	1	94.00	92.16±1.61	208.53±41.96
0.2	6	0.2	0.2	0.9	1	94.43	92.19±1.63	197.13±28.03
0.2	6	0.2	0.2	0.95	1	94.43	92.23±1.61	196.26.00±26.41

5.3 算法性能分析

算法全部参数设置如下：迭代次数 $Iteration$ 为 100，RCL 比例参数 α 为 0.2，状态离散粒度 t 为 1，算子序列长度 $length$ 为 6，Q-learning 探索概率 ε 为 0.2，Q-learning 学习率 β 为 0.2，Q-learning 折扣因子 γ 为 0.95。为消除随机性，每组实验均独立运行十次取平均值作为最终结果。

5.3.1 低层算子消融性实验

为进一步分析算法框架中低层算子设计的有效性，本文从算子扰动规模的角度，对不同算子组合策略进行了消融实验，其中 LLH1-LLH6 归为小规模扰动算子，LLH7-LLH9 归为大规模扰动算子。在统一参数与算法设置和仅依赖低层构造启发式而不采用高层“构造-学习-强化”机制这两组背景下，分别构造三种变体：仅保留小规模扰动算子、仅保留大规模强扰动算子以及同时保留两类算子，共计 6 组实验。各组在 LN02 算例上的平均装载率如表 6 所示。

表 6 低层算子消融实验数据

算法类型	算子类型	平均装载率	算法类型	算子类型	平均装载率
本文算法框架	小规模扰动算子	91.36	仅低层构造启发式	小规模扰动算子	89.76
	大规模扰动算子	90.87		大规模扰动算子	89.74
	保留两类算子	92.23		保留两类算子	89.91

由表 6 可知，在相同算子集合下，引入“构造—学习—强化”机制的超启发式算法可以显著提升装载率，在各算子类型下都超越仅依靠低层构造启发式所得到的平均装载率，证明该机制能够有效挖掘算子潜力，显著改善求解质量。其次，在两种算法下，同时保留小规模和大规模扰动算子明显优于任一单独子集，说明小规模算子提供的局部精修能力与大规模算子带来的解空间跳跃能力具有互补性，并且在超启发式算法的指导下可以形成更为合理的扰动序列配置，从而在强化局部搜索与保持多样性之间取得更好的平衡。

5.3.2 时间复杂度验证实验

由理论推导得到算法时间复杂度为 $T_{total}(n) = O(I \cdot L(A + L)n^2)$ ，为进一步评估算法在实际规模扩展下的计算开销，本文固定了迭代次数 $I = 100$ 、算子数量 $A=9$ 、算子序列长度 $L = 6$ ，可简化时间复杂度为 $O(n^2)$ 。并且构造了货物数量 $n = 100 \sim 1000$ ，步长为 100 的简单算例，箱子的长宽高为 200，货物的长宽高为 10，进行了运行时间统计如表 7 所示。

表 7 不同货物规模的运行时间

货物规模	运行时间	货物规模	运行时间
100	27.58	600	472.81
200	81.53	700	607.09
300	149.78	800	749.08

400	224.72	900	923.98
500	343.44	1000	1001.38

对表 7 实验数据进行对数拟合, 将货物数量和对应的平均运行时间分别取对数, 在对数坐标下得到一组点, 然后在这些点上进行最小二乘线性回归, 得到一条拟合直线。该直线的斜率可以解释为时间随货物规模增长时的幂指数, 得到时间增长指数约为 1.6, 介于 $O(n^{1.5})$ 与 $O(n^2)$ 之间, 说明实际运行时间优于最劣情况下的保守上界, 能够在可以接受的时间范围内求解大规模三维装箱问题。

5.3.3 Martello 算例实验

考虑到三维装箱问题是经典的 NP 难问题, 在问题规模较大的情况下, 精确算法难以在可接受的时间内得到解, 因此实验设置生成的货物数量为 10 个来比较本文算法和 COPT 求解器所得到的最优解之间的差距, 具体如表 8 所示。

表 8 随机生成算例求解对比

算法	COPT	本文算法	差值
class1	87.93	87.93	0
class2	89.36	89.36	0
class3	82.06	79.86	-2.2%
class4	84.3	84.3	0
class5	82.75	77.18	-5.57%
class6	84.9	84.9	0
class7	64.58	64.58	0
class8	69.61	69.61	0

本文算法在 8 个类别中有 6 个类别达到与最优一致, 其余类别的差距处也相对较小, 平均绝对差约 0.97%, 表明在小规模实例上几乎达到了最优的装载性能, 具有较好的寻优能力。

5.3.4 LN 算例实验

LN 算例规模相对较大, 精确算法无法求解, 为评估所提算法的稳健性, 本文对 15 个算例分别独立运行 10 次, 记录每次运行所得的装载率, 计算每个算例的最大值 (Max)、最小值 (Min) 与算术平均值 (Avg), 实验结果如表 9 所示。从表中可以看出, 在最好的情形下, 除算例 LN02 与 LN06 外, 其余 13 个算例均实现了货物完全装载; 即使在最坏的情形下仍有 11 个算例实现货物完全装载。上述结果充分说明所提算法在处理大规模装箱问题时既具备较好的质量, 又具有良好的稳定性。

表 9 本文算法测试对比

装载率	Max	Min	Average	装载率	Max	Min	Average
LN01	62.5	62.5	62.5	LN09	61.9	61.9	61.9
LN02	94.43	89.12	92.23	LN10	67.3	67.3	67.3

LN03	53.4	53.4	53.4	LN11	62.2	62.2	62.2
LN04	55	55	55	LN12	78.5	78.5	78.5
LN05	77.2	77.2	77.2	LN13	85.61	85.09	85.51
LN06	92.67	90.25	91.20	LN14	62.8	62.8	62.8
LN07	84.66	83.74	84.57	LN15	59.5	59.5	59.5
LN08	59.4	59.4	59.4	平均	70.47	69.86	70.21

同时，为验证该算法的性能以及收敛时间，本实验采用较难填充的 LN02 作为例子进行实验测试，LN02 算例货物共装载 179 个货物，装箱效果如图 17（a）所示，由图可以看出，体积较大的货物构成主要骨架，小件则以“砖块式”方式嵌补在行列末端与层间空隙，方便实际装卸作业操作。由算法收敛曲线图 17（b）可知，本文算法在前 20 次迭代时快速提升然后开始收敛，表明算法的收敛性比较好，能以较短的时间求得一个较好的解。

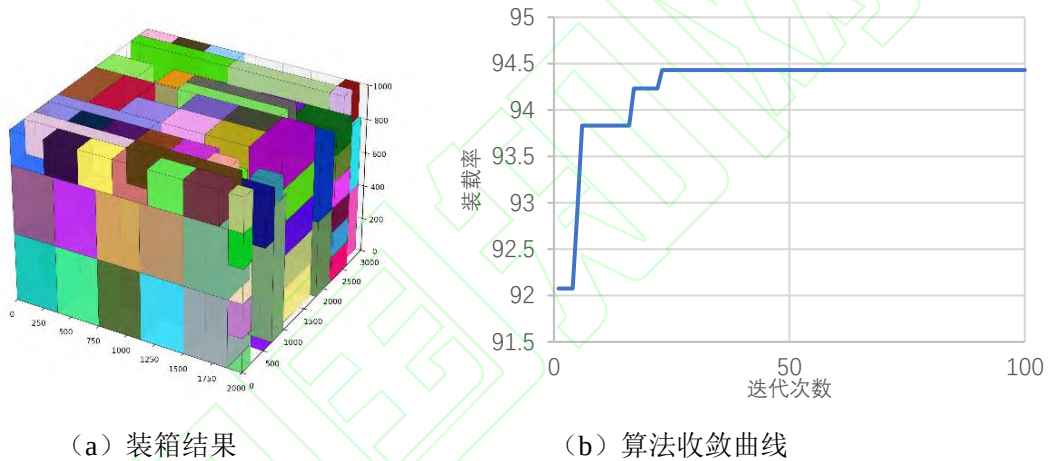


图 17 装箱结果展示图

在 LN 算例上，将本文算法和改进混合遗传模拟退火算法^[26]和进化与局部搜索的混合元启发式算法^[27]进行对比，实验结果如表 10 所示，从结果可以看出，本文方法在大多数算例中装载率优于对比算法，尤其在 LN02、LN06 和 LN13 这三个算例上有较高的提升，总体平均装载率也高于对比算法。其他 LN 算例中，本文算法求得的平均装载率均大于或等于对比算法，说明本文算法在解决三维装箱问题上具有更好性能。

表 10 算法对比

算法	文献[26]	文献[27]	本文方法	算法	文献[26]	文献[27]	本文方法
LN01	62.5	62.5	62.5	LN09	61.9	61.9	61.9
LN02	88.3	87.7	92.23	LN10	67.3	67.3	67.3
LN03	53.4	53.4	53.4	LN11	62.2	62.2	62.2
LN04	55	55	55	LN12	78.5	78.5	78.5
LN05	77.2	77.2	77.2	LN13	79.4	80.5	85.51

LN06	84.8	82.5	91.20	LN14	62.8	62.8	62.8
LN07	82.9	73.2	84.57	LN15	59.5	59.5	59.5
LN08	59.4	59.4	59.4	平均	69.01	68.24	70.21

5.4 企业实例验证

为验证所提出的三维装箱超启发式算法在实际企业场景中的可行性,选取了文献[26]和文献[28]所提供的企业实例数据进行实验验证(下文简称为实例 1、实例 2),具体数据如表 11 所示。

表 11 企业实例数据

文献[26]企业实例数据					文献[28]企业实例数据				
箱子信息	宽/cm	长/cm	高/cm	数量	箱子信息	宽/cm	长/cm	高/cm	数量
	318	224	163	1		1196	235	269	1
货物信息	宽/cm	长/cm	高/cm	数量	货物信息	宽/cm	长/cm	高/cm	数量
	43	21	27	50		43	39	66	500
	53	29	37	150		80	47.5	58	7
	26	15	18	200		86	49	50	1
	53	29	37	60		77	54	44.5	45
	43	21	27	50		28.5	23	23	6
	53	29	37	40		55	43	17	10
	43	21	27	50		36	36	22	139
						44	33	33	4

使用本文算法对两组企业实例进行装载,结果如表 12 所示:在实例 1 中,本文算法共装载了 348 个货物,装载率为 99.45%,与文献[26]的装载率 94.55%相比,提升了约 4.9%装载率。在实例 2 中,本文算法共装载了 712 个货物,装载率为 92.68%,与文献[28]的装载率 91.9%相比,提升了约 0.78%装载率,两组实例装载可视化如图 18 所示。总体而言,两组企业实例较对比方法均取得改进:实例 1 接近箱子装载可达上限,实例 2 在不利的尺寸分布下仍实现货物完全装载,表明了本文算法的实用性。

表 12 实例结果比较

数据类别	文献[26]	本文方法	数据类别	文献[28]	本文方法
实例 1	94.55%	99.45%	实例 2	91.9%	92.68%

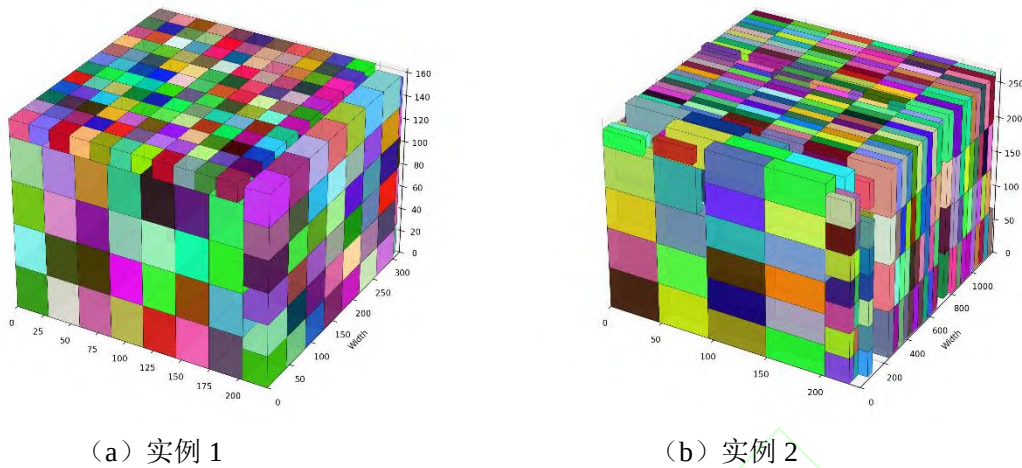


图 18 装箱可视化展示图

6 结束语

为求解三维装箱问题,本文提出了一种基于“构造—学习—强化”的超启发式算法:先由 GRASP 选择算子生成 RCL 实现“构造”,Q-learning 在 RCL 内进行 ϵ -贪婪自适应选择,按需组合九类低层扰动算子生成算子序列并进行“学习”,最后用 PR 沿径强化和局部改进实现“强化”,再经最深左下角与三空间分割解码,得到可行放置方案。为保证评价客观,本文采用 Loh&Nee 以及 Martello 所提出的经典算例以及两组企业实例作为实验数据,并分别与其他已有算法进行比较。实验显示本文方法的收敛速度与稳定性俱佳,且其结果也优于其他算法,逼近精确解,验证了所提算法的有效性与通用性。由于大规模三维装箱问题在有限时间内难以获得最优解,本文方法着重于解决单箱装载的三维装箱问题,因此在更大货物规模、异质性更强的箱型配置以及更复杂的物流装载环境中的表现仍有进一步提升空间。得益于超启发式算法框架的模块化结构,并具有一定的任务无关性,因此未来可通过扩展低层算子体系与构造式放置方法,将本文方法推广至多箱装载、异构容器选择、路径优化以及其他具有序列扰动特征的物流调度任务中,以进一步提升其应用范围与实际工程价值。

参考文献:

- [1] CHAZELLE B, EDELSBRUNNER H, GUIBAS L J. The complexity of cutting complexes[J]. *Discrete & Computational Geometry*, 1989, 4(2): 139-181.
- [2] ROMANOVA T, STOYAN Y, PANKRATOV A, et al. Sparsest balanced packing of irregular 3D objects in a cylindrical container[J]. *European Journal of Operational Research*, 2021, 291(1): 84-100.
- [3] MARTELLO S, PISINGER D, VIGO D. The three-dimensional bin packing problem[J]. *Operations Research*, 2000, 48(2): 256-267.
- [4] TAYLOR G S, CHAN Y, RASOOL G. A three-dimensional bin-packing model: exact multicriteria solution and computational complexity[J]. *Annals of Operations Research*, 2017, 251(1-2): 397-427.
- [5] MAHVASH B, AWASTHI A, CHAUHAN S. A column generation-based heuristic for the three-dimensional bin packing problem with rotation[J]. *Journal of the Operational Research Society*, 2018, 69(1): 78-90.
- [6] LODI A, MARTELLO S, VIGO D. Heuristic algorithms for the three-dimensional bin packing problem[J]. *European Journal of Operational Research*, 2002, 141(2): 410-420.
- [7] BISCHOFF E E, RATCLIFF M S W. Issues in the development of approaches to container loading[J]. *Omega*, 1995, 23(4): 377-390.
- [8] CRAINIC T G, PERBOLI G, TADEI R. Extreme point-based heuristics for three-dimensional bin packing[J]. *INFORMS Journal on Computing*, 2008, 20(3): 368-384.
- [9] FANSLAU T, BORTFELDT A. A tree search algorithm for solving the container loading problem[J]. *INFORMS Journal on Computing*, 2010, 22(2): 222-235.
- [10] CRAINIC T G, PERBOLI G, TADEI R. TS2PACK: A two-level tabu search for the three-dimensional bin packing problem[J]. *European Journal of Operational Research*, 2009, 195(3): 744-760.
- [11] GEHRING H. A genetic algorithm for solving the container loading problem[J]. *International Transactions in Operational Research*, 1997, 4(5-6): 401-418.
- [12] CAGAN J, DEGENTES D, YIN S. A simulated annealing-based algorithm using hierarchical models for general three-dimensional component layout[J]. *Computer-Aided Design*, 1998, 30(10): 781-790.
- [13] ALVAREZ-VALDÉS R, PARREÑO F, TAMARIT J M. A GRASP/Path relinking algorithm for two- and three-dimensional multiple bin-size bin packing problems[J]. *Computers & Operations Research*, 2013, 40(12): 3081-3090.
- [14] FEO T A, RESENDE M G C. A probabilistic heuristic for a computationally difficult set covering problem[J]. *Operations Research Letters*, 1989, 8(2): 67-71.
- [15] ROSS P, MARÍN-BLÁZQUEZ J G, SCHULENBURG S, et al. Learning a Procedure That Can Solve Hard Bin-Packing Problems: A New GA-Based Approach to Hyper-heuristics[C]//CANTÚ-PAZ E, et al. *Genetic and Evolutionary Computation—GECCO 2003. Lecture Notes in Computer Science*, vol 2724. Berlin, Heidelberg: Springer, 2003: 1295-1306.

- [16] BAI R, BLAZEWICZ J, BURKE E K, et al. A simulated annealing hyper-heuristic methodology for flexible decision support[J]. 4OR, 2012, 10(1): 43-66.
- [17] 奎昊,朱荣,胡蓉,等.三阶段优化算法求解带三维装载约束的 MDVRP[J].控制工程,2023,30(11):2027-2040.
- KUI Hao, ZHU Rong, HU Rong, et al. A Three-stage Optimization Algorithm for Solving Three-dimensional Loading-capacitated Multiple Depot Vehicle Routing Problem[J]. Control Engineering of China,2023,30(11):2027-2040.
- [18] 张佳朋,庄存波,刘检华,等.基于超启发式算法的可重构装配车间调度[J].计算机集成制造系统,2025,31(02):399-410.
- ZHANG Jiapeng, ZHUANG Cunbo, LIU Jianhua, et al. Reconfigurable assembly shop scheduling based on hyper heuristics algorithms[J]. Computer Integrated Manufacturing System, 2025, 31(2): 399.
- [19] 王舜,赵燕伟,冷龙龙,等.基于蚁群选择超启发算法的低碳选址—路径问题[J].计算机集成制造系统,2020,26(06):1702-1716.
- WANG Shun, ZHAO Yanwei, LENG Longlong, et al. Low carbon location-routing problem based on evolutionary hyper-heuristic algorithm of ant colony selection mechanism[J]. Computer Integrated Manufacturing System, 2020,26(06):1702-1716.
- [20] WATKINS C J C H, DAYAN P. Q-learning[J]. Machine Learning, 1992, 8(3): 279-292.
- [21] 郭姝娟,彭康真,关泽坤,等.基于 Q 学习超启发算法的不确定环境下多码头集卡路径优化问题[J].系统工程理论与实践,2024,44(12):4097-4112.
- GUO Shujuan, PENG Kangzhen, GUAN Zekun, et al. A Q-learning hyper-heuristic algorithm for inter-terminal truck routing problem with uncertainties[J]. Systems Engineering-Theory & Practice,2024,44(12):4097-4112.
- [22] 崔建双,吕玥,徐子涵.基于 Q 学习的超启发式模型及算法求解多模式资源约束项目调度问题[J].计算机集成制造系统,2022,28(05):1472-1481.
- CUI Jianshuang, LV Yue, XUE Zihan. Q-learning based hyper-heuristic algorithm for solving multi-mode resource-constrained project scheduling problem[J]. Computer Integrated Manufacturing Systems, 2022, 28(5): 1472.
- [23] 张景玲,冯勤炳,赵燕伟,等.基于强化学习的超启发算法求解有容量车辆路径问题[J].计算机集成制造系统,2020,26(04):1118-1129.
- ZHANG Jingling, FENG Qinbing, ZHAO Yanwei, et al. Hyper-heuristic for CVRP with reinforcement learning[J]. Computer Integrated and Manufacturing Systems, 2020, 26(4): 1118-1129.
- [24] MARTELLO S, MONACI M, VIGO D. An exact approach to the strip-packing problem[J]. INFORMS Journal on Computing, 2003, 15(3): 310-319.
- [25] LOH T H. NEE A Y C. A Packing Algorithm for Hexahedral Boxes[C]//Proceedings of the Conference of Industrial Automation. Singapore, 1992, 115-126.
- [26] 张钧,贺可太.求解三维装箱问题的混合遗传模拟退火算法[J].计算机工程与应用,2019,55(14):32-39+47.
- ZHANG Jun, HE Ketai. Study on hybrid genetic and simulated annealing algorithm for three-dimensional packing problems[J]. Computer Engineering and Applications, 2019, 55(14): 32-39.
- [27] 刘永,岳志城,王勇.托盘装载约束下带时间窗的配送车辆路径优化研究[J].交通运输

系统工程与信息,2023,23(06):262-273+295.

LIU Yong, YUE Zhicheng, WANG Yong. Distribution Routing Problem with Time Window Under Constraint of Pallet Loading[J]. Journal of Transportation Systems Engineering and Information Technology,2023,23(06):262-273+295.

[28] 翟钰,孙小明.多种物品三维装箱问题的一种启发式算法[J].上海交通大学学报,2007,(08):1244-1247.

ZHAI Yu, SUN Xiaoming. A heuristic algorithm for three-dimensional container loading problem with non-identical items[J]. JOURNAL-SHANGHAI JIAOTONG UNIVERSITY-CHINESE EDITION-, 2007, 41(8): 1244.

作者简介:

+徐翔斌(1975-),男,江西湖口人,教授,博士,研究方向:三维装箱和车辆配载优化,通讯作者,E-mail: xuxiangbin@ecjtu.edu.cn;

马国飞(2002-),男,浙江嘉兴人,硕士研究生,研究方向:三维装箱,E-mail:930446084@qq.com。